

ГУАП

КАФЕДРА № 42

ОТЧЕТ  
ЗАЩИЩЕН С ОЦЕНКОЙ  
ПРЕПОДАВАТЕЛЬ

ассистент

\_\_\_\_\_  
должность, уч. степень, звание

\_\_\_\_\_  
подпись, дата

И.Д. Свеженин

\_\_\_\_\_  
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

Использование паттернов в нативных  
мобильных приложениях

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

\_\_\_\_\_  
подпись, дата

Г.В. Буренков

\_\_\_\_\_  
инициалы, фамилия

Санкт-Петербург 2025

## СОДЕРЖАНИЕ

|                                    |   |
|------------------------------------|---|
| 1. Задание .....                   | 2 |
| 2. Ход работы.....                 | 3 |
| 3. Результат работы программы..... | 5 |
| 4. Вывод.....                      | 8 |
| 5. Листинг с кодом программы.....  | 9 |

## **1. Задание**

Задание заключалось в разработке системы аутентификации для существующего мобильного приложения под Android на Kotlin с интеграцией Firebase Authentication. Требовалось создать систему регистрации новых пользователей с использованием email и пароля, реализовать функциональность входа в систему с валидацией учетных данных, добавить возможность изменения пароля текущего пользователя с проверкой минимальной длины, реализовать удаление учетной записи с подтверждением операции через диалоговое окно, интегрировать Firebase SDK в проект через Gradle с настройкой google-services plugin, создать пользовательский интерфейс с формами ввода данных и динамическим переключением между состояниями авторизованного и неавторизованного пользователя, а также обеспечить сохранение состояния аутентификации между сессиями приложения с автоматической проверкой текущего пользователя при запуске фрагмента. Дополнительно требовалось интегрировать систему аутентификации в существующую архитектуру приложения через вкладку "Аккаунт" в нижней навигации с использованием фрагментов и обеспечить корректную обработку ошибок Firebase с выводом информативных сообщений пользователю.

## 2. Ход работы

Первый этап включал настройку Firebase в проекте и подключение необходимых зависимостей. В `build.gradle.kts` добавлен `plugin com.google.gms.google-services` и зависимости Firebase BOM, `firebase-auth-ktx` и `firebase-analytics-ktx`. Создан файл `google-services.json` с конфигурацией проекта Firebase. В Firebase Console создан проект, добавлено Android приложение с указанием `package name com.example.autotasks`, включен метод аутентификации Email/Password в разделе Authentication.

Второй этап был посвящен разработке пользовательского интерфейса для системы аутентификации. Создан `layout fragment_account.xml` с использованием `ScrollView` для обеспечения прокрутки контента. Реализованы два контейнера: `layoutLoggedOut` для неавторизованных пользователей с полями для ввода email и пароля, кнопками "Войти" и "Зарегистрироваться", и `layoutLoggedIn` для авторизованных пользователей с отображением email, полем для нового пароля, кнопками "Изменить пароль", "Выйти" и "Удалить аккаунт". Добавлен `TextView` для отображения сообщений с динамической сменой цвета и автоматическим скрыванием.

Третий этап включал реализацию логики аутентификации в `FragmentAccount.kt` с использованием `Firebase Authentication SDK`. Инициализирован `FirebaseAuth` для получения экземпляра сервиса аутентификации. Реализованы методы `registerUser()` и `loginUser()` с валидацией данных, вызовами `createUserWithEmailAndPassword()` и `signInWithEmailAndPassword()`, обработкой результатов через `addOnCompleteListener` и обновлением интерфейса. Добавлен метод `logoutUser()` с вызовом `auth.signOut()` и `updateUI()` для динамического переключения видимости контейнеров в зависимости от состояния аутентификации.

Четвертый этап был посвящен добавлению функций управления учетной записью. Реализован метод `changePassword()` с получением нового пароля, валидацией длины и вызовом `user.updatePassword()` для обновления на

сервере Firebase. Добавлен метод `confirmDeleteAccount()` с созданием `AlertDialog` для подтверждения операции и `deleteAccount()` с вызовом `user.delete()` для полного удаления учетной записи. Реализован метод `showMessage()` для унифицированного вывода сообщений с цветовой индикацией успеха или ошибки и автоматическим скрыванием через три секунды.

### **3. Результат работы программы**

В качестве результата работы программы представлено пару фотографий интерфейса. На рисунках 1-2 показан пример работы программы.

11:11

# Аккаунт

Email

Пароль

ВОЙТИ

ЗАРЕГИСТРИРОВАТЬСЯ

Гонщик

Карта

Аккаунт

Рисунок 1 – Блок входа в приложении

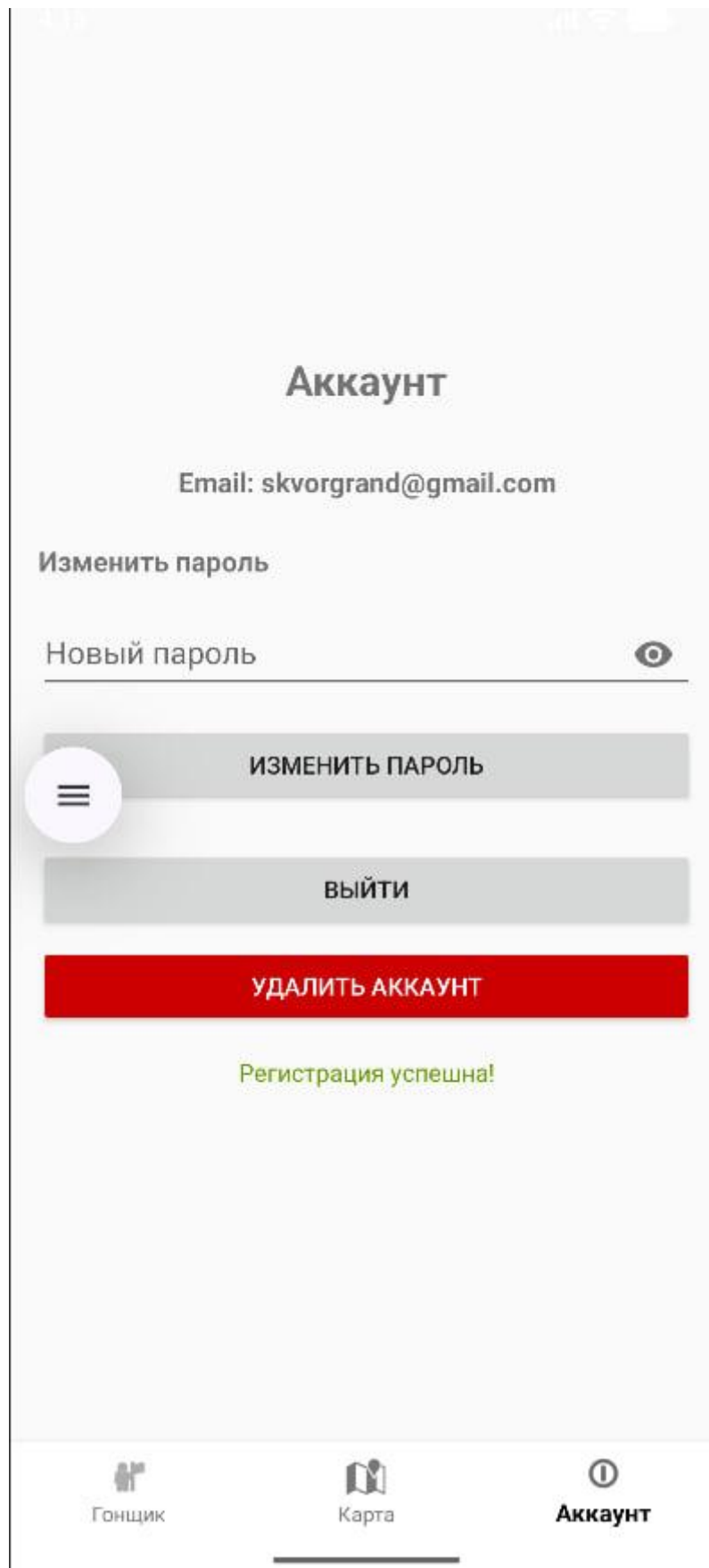


Рисунок 2 – Просмотр действий после регистрации



#### **4. Вывод**

В ходе выполнения лабораторной работы разработана полнофункциональная система аутентификации для мобильного приложения Android на Kotlin с интеграцией Firebase Authentication. Приложение обеспечивает регистрацию новых пользователей с использованием email и пароля, вход в систему с валидацией учетных данных, изменение пароля и удаление учетной записи с подтверждением операции. Использована облачная платформа Firebase для серверной обработки аутентификации, что обеспечивает безопасное хранение паролей и защиту от несанкционированного доступа. Реализована интеграция системы в существующую архитектуру приложения через фрагмент с динамическим управлением состоянием интерфейса.

Выполнение работы позволило освоить интеграцию облачных сервисов Firebase в Android приложения, работу с Firebase Authentication SDK для управления пользователями, создание адаптивного пользовательского интерфейса с динамическим переключением состояний, обработку асинхронных операций через `addOnCompleteListener` и реализацию диалоговых окон для подтверждения критических операций. Разработанное решение демонстрирует практическое применение современных подходов к разработке систем аутентификации в мобильных приложениях и может быть расширено дополнительным функционалом, таким как восстановление пароля через email, аутентификация через социальные сети и двухфакторная аутентификация для повышения безопасности.

## 5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:  
#mainactivity.kt

```
package com.example.autotasks

import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import androidx.fragment.app.commit
import
com.google.android.material.bottomnavigation.BottomNavigationView
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.delay
import org.json.JSONArray
import java.net.HttpURLConnection
import java.net.URL
import android.net.ConnectivityManager
import android.net.NetworkInfo
import android.widget.Toast
import com.example.autotasks.database.AppDatabase
import com.example.autotasks.database.Driver
import com.example.autotasks.network.SupabaseService
import com.example.autotasks.network.SupabaseSync
```

```

import
com.google.android.material.floatingactionbutton.FloatingActionButton

@Suppress("DEPRECATION")
class MainActivity : AppCompatActivity() {

    val drivers = mutableListOf<Driver>()
    var currentDriverIndex = 0
    var lastLoadTime: Long = 0 // Время последней загрузки данных

    private lateinit var fragmentDriver: FragmentDriver
    private lateinit var database: AppDatabase

    companion object {
        const val REQUEST_ADD_DRIVER = 1
        const val REQUEST_EDIT_DRIVER = 2
    }

    private fun isNetworkAvailable(): Boolean {
        val connectivityManager =
getSystemService(CONNECTIVITY_SERVICE) as ConnectivityManager
        val networkInfo: NetworkInfo? =
connectivityManager.activeNetworkInfo
        return networkInfo?.isConnected == true
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

```

```

        database = AppDatabase.getDatabase(this)

        if (!isNetworkAvailable()) {
            Toast.makeText(this, "Нет интернета",
                Toast.LENGTH_SHORT).show()
        }

        val bottomNav =
            findViewById<BottomNavigationView>(R.id.bottomNavigation)
        val fabAdd = findViewById<FloatingActionButton>(R.id.fabAdd)
        val fabRefresh =
            findViewById<FloatingActionButton>(R.id.fabRefresh)

        lifecycleScope.launch {
            // Сначала загружаем из локальной БД для быстрого отображения
            loadDriversFromDatabase()

            // Убеждаемся, что lastLoadTime установлен
            if (lastLoadTime == 0L) {
                lastLoadTime = System.currentTimeMillis()
                android.util.Log.w("MainActivity", "lastLoadTime не был
установлен, устанавливаем вручную")
            }

            // Создаем фрагмент гонщиков сразу, чтобы UI не был пустым
            fragmentDriver = FragmentDriver.newInstance(
                drivers,
                ::loadDriverInfo,
                ::showDriverActionsDialog,
                ::refreshDrivers,

```

```

        { lastLoadTime } // Функция для получения времени последней
загрузки
    )
    supportFragmentManager.commit {
        replace(R.id.fragmentContainer, fragmentDriver)
    }

    // Обновляем фрагмент после создания, чтобы показать время
загрузки
    kotlinx.coroutines.delay(100) // Даем время adapter
инициализироваться
    if (::fragmentDriver.isInitialized) {
        fragmentDriver.refreshDrivers()
    }
}

    // Синхронизация с Supabase при запуске - загружаем актуальные
данные из реальной БД
    Thread {
        try {
            // Небольшая задержка, чтобы UI успел отобразиться
            Thread.sleep(500)

            if (isNetworkAvailable()) {
                android.util.Log.d("MainActivity", "Начинаем синхронизацию
с Supabase...")
                runBlocking {
                    try {
                        // Загружаем актуальные данные из Supabase
                        loadDriversFromSupabaseSync()

```

```

        android.util.Log.d("MainActivity", "Синхронизация с
Supabase завершена")

        // Перегружаем из локальной БД (которая теперь
обновлена)

        loadDriversFromDatabase()

        // Обновляем UI
        withContext(Dispatchers.Main) {
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    } catch (e: Exception) {
        android.util.Log.e("MainActivity", "Ошибка
синхронизации с Supabase: ${e.message}")
        e.printStackTrace()
        withContext(Dispatchers.Main) {
            // В случае ошибки просто загружаем из локальной
БД

            lifecycleScope.launch {
                loadDriversFromDatabase()
                if (::fragmentDriver.isInitialized) {
                    fragmentDriver.refreshDrivers()
                }
            }
        }
    }
} else {

```

```

        android.util.Log.w("MainActivity", "Нет интернета,
используем только локальную БД")
    }
} catch (e: Exception) {
    android.util.Log.e("MainActivity", "Ошибка в потоке
синхронизации: ${e.message}")
    e.printStackTrace()
}
}.start()

```

```

fabAdd.setOnClickListener {
    val intent = Intent(this, AddDriverActivity::class.java)
    startActivityForResult(intent, REQUEST_ADD_DRIVER)
}

```

```

fabRefresh.setOnClickListener {
    // Обновляем данные из Supabase и обновляем список
    refreshFromSupabase()
}

```

```

bottomNav.setOnItemSelectedListener { item ->
    when (item.itemId) {
        R.id.menu_driver -> {
            // фрагмент гонщиков
            supportFragmentManager.commit {
                replace(R.id.fragmentContainer, fragmentDriver)
            }
            true
        }
        R.id.menu_cards -> {

```

```

        // Показываем фрагмент карт
        supportFragmentManager.commit {
            replace(
                R.id.fragmentContainer,
                FragmentCar.newInstance(::loadCarInfo,
currentDriverIndex, "")
            )
        }
        true
    }
    R.id.menu_account -> {
        // Показываем фрагмент аккаунта
        supportFragmentManager.commit {
            replace(R.id.fragmentContainer,
FragmentAccount.newInstance())
        }
        true
    }
    else -> false
}
}
}

```

```

// Загружаем гонщиков из БД
private suspend fun loadDriversFromDatabase() {
    withContext(Dispatchers.IO) {
        try {

```



```

        // Добавляем небольшую задержку для гарантии реальной
загрузки из БД
        kotlinx.coroutines.delay(100)

        // Сначала проверяем количество записей в БД
        val count = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Количество записей в БД
(COUNT): $count")

        // Затем загружаем все записи - принудительно, без кэша
        val dbDrivers = database.driverDao().getAllDrivers()

        android.util.Log.d("MainActivity", "Загружено из БД:
${dbDrivers.size} гонщиков")
        android.util.Log.d("MainActivity", "Время загрузки:
${System.currentTimeMillis()}")

        // Проверяем, что количество совпадает
        if (count != dbDrivers.size) {
            android.util.Log.e("MainActivity", "ОШИБКА:
Несоответствие! COUNT=$count, getAllDrivers=${dbDrivers.size}")
        }

        // Логируем детали загруженных гонщиков
        if (dbDrivers.isNotEmpty()) {
            android.util.Log.d("MainActivity", "ID загруженных
гонщиков: ${dbDrivers.map { it.id }}" )
            android.util.Log.d("MainActivity", "Номера гонщиков:
${dbDrivers.map { it.driverNumber }}" )
        } else {

```

```
        android.util.Log.w("MainActivity", "ВНИМАНИЕ: БД  
вернула пустой список!")  
    }
```

```
    withContext(Dispatchers.Main) {  
        val oldSize = drivers.size  
        val oldIds = drivers.map { it.id }.toSet()  
  
        // Полностью очищаем список перед добавлением новых  
        данных
```

```
        drivers.clear()
```

```
        // Создаем новый список из загруженных данных  
        val newDriversList = dbDrivers.toMutableList()  
        drivers.addAll(newDriversList)
```

```
        val newIds = drivers.map { it.id }.toSet()
```

```
        // Обновляем время последней загрузки  
        lastLoadTime = System.currentTimeMillis()
```

```
        android.util.Log.d("MainActivity", "Список обновлен. Старый  
размер: $oldSize, Новый размер: ${drivers.size}")
```

```
        // Логируем изменения  
        if (oldIds != newIds) {  
            val removed = oldIds - newIds  
            val added = newIds - oldIds  
            if (removed.isNotEmpty()) {
```

```

        android.util.Log.d("MainActivity", "Удалены ID:
$removed")
    }
    if (added.isNotEmpty()) {
        android.util.Log.d("MainActivity", "Добавлены ID:
$dadded")
    }
}

// Если список стал пустым, но был не пустым - это проблема
if (oldSize > 0 && drivers.isEmpty()) {
    android.util.Log.w("MainActivity", "ВНИМАНИЕ: Список
стал пустым после загрузки из БД! Старый размер: $oldSize")
}

// Если размер не совпадает с БД - это проблема
if (drivers.size != dbDrivers.size) {
    android.util.Log.e("MainActivity", "ОШИБКА: Размер
списка не совпадает с БД! Список: ${drivers.size}, БД: ${dbDrivers.size}")
}
}
} catch (e: Exception) {
    e.printStackTrace()
    android.util.Log.e("MainActivity", "Ошибка загрузки из БД:
${e.message}")
    // В случае ошибки не очищаем список
    withContext(Dispatchers.Main) {
        android.util.Log.w("MainActivity", "Ошибка загрузки,
сохраняем текущий список (размер: ${drivers.size})")
    }
}

```

```

        }
    }
}

// Обновление данных из Supabase (для кнопки refresh)
private fun refreshFromSupabase() {
    if (!isNetworkAvailable()) {
        Toast.makeText(this, "Нет интернета",
            Toast.LENGTH_SHORT).show()
        return
    }

    // Показываем индикатор загрузки
    Toast.makeText(this, "Обновление данных...",
        Toast.LENGTH_SHORT).show()

    Thread {
        try {
            runBlocking {
                android.util.Log.d("MainActivity", "Начало обновления из
Supabase...")

                // Загружаем актуальные данные из Supabase
                loadDriversFromSupabaseSync()
                android.util.Log.d("MainActivity", "Синхронизация с
Supabase завершена")

                // Перезагружаем из локальной БД (которая теперь
обновлена)
                loadDriversFromDatabase()

```

```

        // Обновляем UI
        withContext(Dispatchers.Main) {
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
            Toast.makeText(this@MainActivity, "Данные обновлены",
Toast.LENGTH_SHORT).show()
        }
    }
} catch (e: Exception) {
    android.util.Log.e("MainActivity", "Ошибка обновления из
Supabase: ${e.message}")
    e.printStackTrace()
    runOnUiThread {
        Toast.makeText(this@MainActivity, "Ошибка обновления:
${e.message}", Toast.LENGTH_SHORT).show()

        // В случае ошибки просто загружаем из локальной БД
        lifecycleScope.launch {
            loadDriversFromDatabase()
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    }
}
}.start()
}

```

// Загружаем гонциков из Supabase синхронно (для первого запуска)

```

private suspend fun loadDriversFromSupabaseSync() {
    withContext(Dispatchers.IO) {
        try {
            val response = SupabaseService.api.getAllDrivers(
                SupabaseService.API_KEY,
                SupabaseService.getAuthorizationHeader()
            )

            if (response.isSuccessful && response.body() != null) {
                val supabaseDrivers = response.body()!!

                // Сохраняем в БД с проверкой дубликатов
                supabaseDrivers.forEach { supDriver ->
                    // Проверяем, существует ли уже гонщик с таким номером
                    val existing =
                        database.driverDao().getDriverByNumber(supDriver.driverNumber)

                    if (existing == null) {
                        // Если нет - добавляем
                        val driver = Driver(
                            fullName = supDriver.fullName,
                            driverNumber = supDriver.driverNumber,
                            firstName = supDriver.firstName,
                            lastName = supDriver.lastName,
                            teamName = supDriver.teamName,
                            teamColour = supDriver.teamColour,
                            nameAcronym = supDriver.nameAcronym,
                            countryCode = supDriver.countryCode,
                            broadcastName = supDriver.broadcastName
                        )
                    }
                }
            }
        }
    }
}

```

```

        database.driverDao().insertDriver(driver)
    } else {
        // Если есть - обновляем
        val driver = Driver(
            id = existing.id,
            fullName = supDriver.fullName,
            driverNumber = supDriver.driverNumber,
            firstName = supDriver.firstName,
            lastName = supDriver.lastName,
            teamName = supDriver.teamName,
            teamColour = supDriver.teamColour,
            nameAcronym = supDriver.nameAcronym,
            countryCode = supDriver.countryCode,
            broadcastName = supDriver.broadcastName
        )
        database.driverDao().updateDriver(driver)
    }
}

// Не обновляем список здесь - это будет сделано после
синхронизации
}
} catch (e: Exception) {
    e.printStackTrace()
}
}
}

// Загружаем гонщиков из Supabase в отдельном потоке (для фоновой
синхронизации)
private fun loadDriversFromSupabase() {

```

```

Thread {
    try {
        val response = runBlocking {
            SupabaseService.api.getAllDrivers(
                SupabaseService.API_KEY,
                SupabaseService.getAuthorizationHeader()
            )
        }

        if (response.isSuccessful && response.body() != null) {
            val supabaseDrivers = response.body()!!

            // Сохраняем в БД с проверкой дубликатов
            runBlocking {
                supabaseDrivers.forEach { supDriver ->
                    val existing =
                        database.driverDao().getDriverByNumber(supDriver.driverNumber)

                    if (existing == null) {
                        val driver = Driver(
                            fullName = supDriver.fullName,
                            driverNumber = supDriver.driverNumber,
                            firstName = supDriver.firstName,
                            lastName = supDriver.lastName,
                            teamName = supDriver.teamName,
                            teamColour = supDriver.teamColour,
                            nameAcronym = supDriver.nameAcronym,
                            countryCode = supDriver.countryCode,
                            broadcastName = supDriver.broadcastName
                        )

```



```

        database.driverDao().insertDriver(driver)
    } else {
        val driver = Driver(
            id = existing.id,
            fullName = supDriver.fullName,
            driverNumber = supDriver.driverNumber,
            firstName = supDriver.firstName,
            lastName = supDriver.lastName,
            teamName = supDriver.teamName,
            teamColour = supDriver.teamColour,
            nameAcronym = supDriver.nameAcronym,
            countryCode = supDriver.countryCode,
            broadcastName = supDriver.broadcastName
        )
        database.driverDao().updateDriver(driver)
    }
}

// Обновляем список
runOnUiThread {
    lifecycleScope.launch {
        loadDriversFromDatabase()
        if (::fragmentDriver.isInitialized) {
            fragmentDriver.refreshDrivers()
        }
    }
}

} catch (e: Exception) {

```

```

        e.printStackTrace()
    }
}.start()
}

private fun refreshDrivers() {
    lifecycleScope.launch {
        try {
            // Загружаем данные из БД
            loadDriversFromDatabase()

            // Убеждаемся, что данные загружены перед обновлением
фрагмента

            // Небольшая задержка для гарантии, что данные в списке
обновлены

            kotlinx.coroutines.delay(50)

            // Обновляем фрагмент после загрузки данных
            if (::fragmentDriver.isInitialized && drivers.isNotEmpty()) {
                fragmentDriver.refreshDrivers()
            } else if (::fragmentDriver.isInitialized) {
                // Даже если список пустой, обновляем фрагмент
                fragmentDriver.refreshDrivers()
            }
        } catch (e: Exception) {
            e.printStackTrace()
            // В случае ошибки все равно пытаемся обновить фрагмент
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    }
}

```

```

    }
}

// Диалог выбора действий
private fun showDriverActionsDialog(driver: Driver) {
    val options = arrayOf("Просмотр", "Обновление", "Удаление")

    AlertDialog.Builder(this)
        .setTitle("Выберите действие")
        .setItems(options) { _, which ->
            when (which) {
                0 -> showDriverDetails(driver)
                1 -> editDriver(driver)
                2 -> confirmDeleteDriver(driver)
            }
        }
        .show()
}

private fun showDriverDetails(driver: Driver) {
    val details = """
        Полное имя: ${driver.fullName}
        Имя: ${driver.firstName}
        Фамилия: ${driver.lastName}
        Номер гонщика: ${driver.driverNumber}
        Команда: ${driver.teamName}
        Цвет команды: #${driver.teamColour}
        Акроним: ${driver.nameAcronym}
        Страна: ${driver.countryCode}
    """
}

```

```
Broadcast Name: ${driver.broadcastName}  
""".trimIndent()
```

```
AlertDialog.Builder(this)  
    .setTitle("Информация о гонщике")  
    .setMessage(details)  
    .setPositiveButton("OK", null)  
    .show()  
}
```

```
private fun editDriver(driver: Driver) {  
    val intent = Intent(this, EditDriverActivity::class.java)  
    intent.putExtra("DRIVER_ID", driver.id)  
    startActivityForResult(intent, REQUEST_EDIT_DRIVER)  
}
```

```
private fun confirmDeleteDriver(driver: Driver) {  
    AlertDialog.Builder(this)  
        .setTitle("Удаление гонщика")  
        .setMessage("Вы уверены, что хотите удалить  
${driver.fullName}?")  
        .setPositiveButton("Да") { _, _ ->  
            deleteDriver(driver)  
        }  
        .setNegativeButton("Нет", null)  
        .show()  
}
```

```
private fun deleteDriver(driver: Driver) {  
    lifecycleScope.launch {
```

```

try {
    val driverNumber = driver.driverNumber
    val driverId = driver.id

    // Удаляем из БД
    withContext(Dispatchers.IO) {
        // Проверяем количество ДО удаления
        val countBefore = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Удаление: количество ДО
удаления: $countBefore")

        database.driverDao().deleteDriver(driver)

        // Проверяем количество ПОСЛЕ удаления
        val countAfter = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Удаление: количество
ПОСЛЕ удаления: $countAfter")

        // Проверяем, что действительно удалилось
        val deleted = database.driverDao().getDriverById(driverId)
        if (deleted != null) {
            android.util.Log.e("MainActivity", "ОШИБКА: Гонщик не
удален из БД! ID: $driverId")
        } else {
            android.util.Log.d("MainActivity", "Гонщик успешно
удален из БД. ID: $driverId, Number: $driverNumber")
        }

        // Проверяем, что количество уменьшилось
        if (countAfter >= countBefore) {

```

```

        android.util.Log.e("MainActivity", "ОШИБКА: Количество
не уменьшилось! Было: $countBefore, Стало: $countAfter")
    }
}

// Синхронизируем удаление с Supabase
SupabaseSync.syncDeleteDriver(driverNumber)

// Принудительно перезагружаем список из БД
loadDriversFromDatabase()

// Удаляем из локального списка на всякий случай
drivers.removeAll { it.id == driverId }

Toast.makeText(this@MainActivity, "Гонщик удален",
Toast.LENGTH_SHORT).show()

// Обновляем фрагмент
if (::fragmentDriver.isInitialized) {
    fragmentDriver.refreshDrivers()
}
} catch (e: Exception) {
    e.printStackTrace()
    Toast.makeText(this@MainActivity, "Ошибка удаления:
${e.message}", Toast.LENGTH_SHORT).show()
}
}
}

```

```

override fun onActivityResult(requestCode: Int, resultCode: Int, data:
Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (resultCode == RESULT_OK) {
        when (requestCode) {
            REQUEST_ADD_DRIVER, REQUEST_EDIT_DRIVER -> {
                // Сразу обновляем список - данные уже должны быть
сохранены в EditDriverActivity
                refreshDrivers()
            }
        }
    }
}

// Получаем информацию о выбранном гонщике из БД
private suspend fun loadDriverInfo(driver: Driver): String {
    return """
        Broadcast Name: ${driver.broadcastName}
        Полное имя: ${driver.fullName}
        Имя: ${driver.firstName}
        Фамилия: ${driver.lastName}
        Номер гонщика: ${driver.driverNumber}
        Команда: ${driver.teamName}
        Цвет команды: #${driver.teamColour}
        Акроним: ${driver.nameAcronym}
        Страна: ${driver.countryCode}
    """.trimIndent()
}

// Получаем данные автомобиля по номеру гонщика (из старого API)
private suspend fun loadCarInfo(driverNumber: Int): String {

```

```

return withContext(Dispatchers.IO) {
    try {
        val url =
            URL("https://api.openf1.org/v1/car_data?driver_number=$driverNumber&session
            _key=9159")

        val conn = url.openConnection() as HttpURLConnection
        conn.requestMethod = "GET"
        conn.connectTimeout = 5000
        conn.readTimeout = 5000

        val code = conn.responseCode
        if (code != 200) return@withContext "Ошибка запроса: $code"

        val data = conn.inputStream.bufferedReader().readText()
        val json = JSONArray(data)
        if (json.length() == 0) return@withContext "Данные автомобиля
        не найдены"

        val obj = json.getJSONObject(0)
        val speed = obj.optInt("speed")
        val nGear = obj.optInt("n_gear")
        val drs = obj.optInt("drs")
        val throttle = obj.optInt("throttle")
        val brake = obj.optInt("brake")
        val rpm = obj.optInt("rpm")
        val date = obj.optString("date")

        return@withContext """"
        Дата: $date
    }
}

```



Скорость: \$speed км/ч

Передача: \$nGear

DRS: \$drs

Газ: \$throttle%

Тормоз: \$brake

RPM: \$rpm

""".trimIndent()

} catch (e: Exception) {

return@withContext "Ошибка: \${e.message}"

}

}

}

}